Process & Decision Documentation

Project/Assignment Decisions

Side Quests and A4 (Individual Work)

Keep this section brief, typically 2 to 4 sentences.

- Made the screen shake when character landed
- Added provided sfx into correct scenarios

Role-Based Process Evidence

Entry Header

Name: Katherine Chen
Role(s): Code editor and AI prompter
Primary responsibility for this work: Using AI to implement Side Quest 5's sound and physics requirements

Goal of Work Session

Briefly describe what you were trying to accomplish during this phase of the assignment.
Examples:

- Add physics-triggered visual effects (dust particles on landing)
- Add contextual sound effects tied to player actions
- Implement background music
- Add a screen shake bonus effect on landing

Tools, Resources, or Inputs Used

- GenAI tools (if used):
    - Claude Sonnet 4.6

GenAI Documentation

If GenAI was used (keep each response as brief as possible):

Date Used: March 13, 2026

Tool Disclosure: Claude Sonnet 4.6

Purpose of Use: Faster implementation of my invisioned physics and sound effects

Summary of Interaction: I told GenAI what sort of vision I had, what sound effects to implement when and what landing physics I wanted to see, and it wrote it into my code

Human Decision Point(s): The core idea of the physics and soud effect placement came from me. I also directly went into the code and edited some small parts (eg. changed around the sound effects used)

Integrity & Verification Note: Since I personally know HTML, JavaScript, and CSS, I was able to personally verify most of which the code wrote and understood that it was mostly correct. This

sidequest was rather simple and straightforward, like jump effects are obviously supposed to be layered on top of a player's jumping state, so there wasn't much confusion on any end.

Scope of GenAI Use: There was only a limited around of sprites to use, so I didn't do much on that end (even though I wanted to make the character glow green when leaf was collected like how it glows red when it gets hit). Instead I just made it so that it looked like my avatar "ate" the leaf and turned green.

Limitations or Misfires: The tool implemented most of what I said correctly, however, there were a lot of files this time, so it would sometimes get asset names wrong and I had to personally go in to change them.

Summary of Process (Human + Tool)

- I identified which player actions needed audio/visual feedback
- I used AI to make my visions a reality
- I made direct decisions about effect intensity and which sounds mapped to which actions

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Turning the fox's white fur green when leaf was rescued
- Having the screen shake during landing

Verification & Judgement

Explain how you evaluated whether your decision or change was appropriate:
Examples:

- Playtesting on live server many times before publishing to github. Also playtested on github's links to make sure everything was working as different platforms take into consideration different issues (uppercase variables, ect.)
- Cross referenced with my mental vision

Limitations, Dead Ends, or Open Questions

Note anything that:

- I didn't have the time or resources to make my own sprites, so there were some ideas that I had that couldn't be implemented
- I also didn't have time to check out certain other sound libraries to import from ourside of the sfx provided, so when I have more time I would've liked to make the landing sfx different from the getting hit sfx

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.
Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

Prompt	Answer

[attatched week 6 example 5 files] For “Add sound or simple physics to your sketch e.g., visual ehects triggered by motion, collisions, or player actions.Bonus: Combine both: use sound andphysics for a reactive, multi-sensory ehect.” Give me some guidance on how to utilize the prexisting sound files to trigger mechanisms like jump sounds, landing animations/states, and screen shake.

Let me read your actual code carefully before giving you the exact steps.Good — I can see you didn't upload the `/src/world/` files (PlayerController, TileBuilder, BoarSystem, etc.) but I have everything I need from `Level.js`, `Game.js`, `SoundManager.js`, and `main.js`. Here is the **exact, minimal set of changes** for the assignment:

What You're Adding

Requirement	Implementation
Sound	Jump sound + landing thud via `SoundManager`
Physics visual effect	Dust particle burst when player lands (physics-triggered)
Bonus: multi-sensory	Screen shake + sound + particles together on hard landing

These are **4 small edits across 3 files**. Nothing gets restructured.

Step 1 — Load the sounds in `main.js`

In `main.js`, find the `boot()` function. Right **after** `soundManager = new SoundManager()`, add two lines:

```
// --- Audio registry ---
soundManager = new SoundManager();

// ADD THESE TWO LINES:
soundManager.load("jump",
"assets/audio/jump.wav");
soundManager.load("land",
"assets/audio/land.wav");
```

Where to get the audio files: Your `index.html` credits list Brackeys' platformer bundle — it contains both a jump and land/footstep SFX. Download it, rename two files `jump.wav` and `land.wav`, and place them in your `assets/audio/` folder.

Step 2 — Add screen shake state to `main.js`

Near the top of `main.js`, right after the other `let` declarations (around line 60), add:

```
// Screen shake state (VIEW)
let shakeFrames = 0;
let shakeAmt = 0;
```

Then inside the `draw()` function, right **after** the `background(...)` call and **before** `parallax?.draw(...)`, add the shake offset:

```
background(bg[0], bg[1], bg[2]);

// ADD THIS BLOCK:
if (shakeFrames > 0) {
  camera.off();
  translate(random(-shakeAmt, shakeAmt), random(-
shakeAmt, shakeAmt));
  camera.on();
  shakeFrames--;
}
```

Then expose a trigger function (add this near the bottom of `main.js`, before `window.setup = setup`):

```
// Expose for PlayerController to call
window.triggerScreenShake = function(amt = 3,
frames = 5) {
  shakeAmt = amt;
  shakeFrames = frames;
};
```

Step 3 — Wire the jump sound in `src/world/PlayerController.js`

Find where your player jumps — look for where `vel.y` is set to a negative value with the jump strength. It will look something like:

```
if (input.jumpPressed && this.sprite.isGrounded) {
  this.sprite.vel.y = -jumpStrength;
```

Add one line immediately after the velocity is set:

```
if (input.jumpPressed && this.sprite.isGrounded) {
  this.sprite.vel.y = -jumpStrength;
```

```
  this.events?.emit("player:jumped");   // ADD
THIS
```

Then in **`Game.js`**, inside `_wireEventListeners()`, alongside the other `this.sound` listeners, add:

```
if (this.sound) {

this._unsubs.push(this.events.on("leaf:collected",
() => this.sound.play("leaf")));

this._unsubs.push(this.events.on("player:damaged",
() => this.sound.play("hurt")));
  this._unsubs.push(this.events.on("player:died",
() => this.sound.play("die")));
  this._unsubs.push(this.events.on("level:won",
() => this.sound.play("win")));
  this._unsubs.push(this.events.on("boar:damaged",
() => this.sound.play("hit")));

this._unsubs.push(this.events.on("player:jumped",
() => this.sound.play("jump")));  // ADD THIS
}
```

Step 4 — Detect landing and spawn dust particles in `src/world/PlayerController.js`

In your `PlayerController`, add two things:

A) Track airborne state — in the `update()` method, at the very top before other logic:

```
update({ input, solids, bounds, won }) {
  // ADD THESE 3 LINES at the very top:
  const wasAirborne = this._airborne ?? false;
  this._airborne = !this.sprite.isGrounded;
  const justLanded = wasAirborne
&& !this._airborne;

  // ... rest of your existing update code ...
```

B) Trigger the effect when landing — still inside `update()`, right after those 3 lines:

```
  if (justLanded) {
    this._spawnDust();
    this.events?.emit("player:landed");
  }
```

C) Add the `_spawnDust()` method to the `PlayerController` class (add it below `update()`):

```
_spawnDust() {
  for (let i = 0; i < 5; i++) {
    const dust = new Sprite(
      this.sprite.x + random(-8, 8),
      this.sprite.y + (this.sprite.h ?? 6),
      3, 3, 'none'           // 'none' collider =
pure visual
    );
    dust.vel.x = random(-1.5, 1.5);
    dust.vel.y = random(-2.5, -0.5);
    dust.color = color(210, 190, 150, 200);
    dust.stroke = 'none';
    dust.life = 18;          // p5play auto-
removes after N frames
  }
  // Bonus: trigger screen shake + landing sound
  window.triggerScreenShake?.(2, 4);
  this.events?.emit("player:landed");
}
```

Step 5 — Wire the landing sound in `Game.js`

In `_wireEventListeners()`, add one more line alongside the others:

```
this._unsubs.push(this.events.on("player:landed",
() => this.sound.play("land")));
```

Summary of all file changes

File	What you added
`main.js`	Load 2 sounds, shake state variables, shake in `draw()`, `triggerScreenShake()` helper
`src/world/PlayerController.js`	Emit `player:jumped`, track `_airborne`, call `_spawnDust()`, add `_spawnDust()` method
`src/Game.js`	Subscribe to `player:jumped` and `player:landed` events → play sounds

	That's it — no new files, no restructuring. The EventBus you already have handles all the decoupling cleanly, and `SoundManager` already has `load()` and `play()` ready to go.
I don’t understand the playercontroller.js additions	Since you didn't upload the `/src/world/` folder, let me check what files you actually have.You never uploaded it — `PlayerController.js` lives in your `src/world/` folder on your computer but wasn't included in any of your uploads. To get the full rewrite, just drag and drop `src/world/PlayerController.js` into the chat and I'll rewrite it completely for you.
[upload playercontroller.js] Here is the full file, help me implement the changes written previously.	Perfect, I have it. I can see `player:jumped` is **already emitted** on line ~55. I just need to add the landing detection and dust effect.Drop this into `src/world/PlayerController.js`. Here's exactly what changed from your original: • **Constructor** — added `this._wasAirborne = false` • **`reset()`** — resets `_wasAirborne` so landing doesn't misfire after restart • **`update()`** — 4 new lines at the top: compare `_wasAirborne` vs current `grounded`, call `_onLand()` if just touched down, then update `_wasAirborne` • **`_onLand()`** — new method: spawns 5 dust sprites, emits `player:landed` (which `Game.js` already listens for now), calls `window.triggerScreenShake()` Everything else in the file is identical to what you pasted.
Next step is to add the p5.js sounds. Here are the [list] of sfx that are available. I would like to use it when appropriate (jump.wav when i jump, ect.)	Your SFX are in `assets/sfx/` and the filenames are slightly different from what I assumed. Let me update the two files that reference sound paths/keys.Your available files are: `hitEnemy.wav`, `jump.wav`, `leafCollect.wav`, `music.wav`, `receiveDamage.wav`. No land sound exists, so I'll map everything correctly and skip `land`. Let me update both files.The uploaded `main.js` doesn't have the sound loads yet — it's the original. I'll add everything fresh.Now add the shake state and `triggerScreenShake` to main.js:Now fix `Game.js` — update the sound event map to match the real keys, and add

<table>
<tr>
<td></td>
<td>

`player:jumped` + `player:landed`:Here's the full sound mapping using your actual files:

Event	Sound file	Trigger
`player:jumped`	`jump.wav`	Player jumps
`leaf:collected`	`leafCollect.wav`	Picks up a leaf
`boar:damaged`	`hitEnemy.wav`	Player attacks a boar
`player:damaged`	`receiveDamage.wav`	Player takes damage
`player:died`	`receiveDamage.wav`	Player dies (no dedicated file)
`level:won`	`leafCollect.wav`	Level complete (no dedicated file)
`player:landed`	`receiveDamage.wav`	Lands after jump (closest thud)

The `PlayerController.js` from the previous message is still correct and unchanged — use that too. Once you have dedicated `die.wav` or `win.wav` files, just swap the paths in `main.js`.

</td>
</tr>
<tr>
<td>

wait how do i attack a boar? also the music is not implemented, can you make music.wav the background music playing on. loop

</td>
<td>

Good questions. Let me check both — attack input and whether music is wired anywhere.Two clear answers:

How to attack a boar: Press **Spacebar**. In `InputManager.js` line 65: `const attackDown = kb.pressing("space")` — that's your attack key. Walk up to a boar and hit Space.

Music: It's completely unimplemented — `music.wav` is never loaded or played anywhere in your codebase. Here's the fix. Only `main.js` needs to change:Now add the shake state and screen shake block (same as before, since this is the original uploaded file):Now update `SoundManager.js` to support looping music:**Attack a boar:** Walk up to one and press **Spacebar**. That's it — it's already wired in `InputManager.js`.

Music fix — 3 files changed:

- **`SoundManager.js`** — added `loadLoop()` and `playLoop()` / `stopLoop()` methods for looping tracks
- **`main.js`** — added `soundManager.loadLoop("music", "assets/sfx/music.wav")` in boot, and `soundManager?.playLoop("music")` inside

</td>
</tr>
</table>

	`unlockAudioOnce()` so it starts on the player's first keypress or click (required by browsers — audio can't autoplay) • **`Game.js`** — added `player:jumped` and `player:landed` event listeners (same as before) The music will kick in the moment the player presses any key or clicks, then loop forever in the background.
is it possible to make the character "glow" or have an outer effect when a leaf is rescued?	Yes, absolutely. You'd emit an event when a leaf is collected (already done — `leaf:collected` fires in `Level.js`) and have `PlayerController` listen for it and set a short glow timer. I can see exactly how the tint system works (`applyHurtBlinkTint` already uses `sprite.tint`). The glow effect just needs a `glowTimer` that pulses a golden tint, slotted into the same tint logic. Only **two files** need changes. Ran a command, created a file Ran a command, created a file Now add one line to `Game.js` to call `triggerLeafGlow()` when the event fires: Edited a file, ran a command, read a file Edited a file, ran a command, read a file Two files, minimal changes: **`PlayerEntity.js`** — four additions: • `this.glowTimer = 0` and `this.GLOW_FRAMES = 50` in the constructor • `this.glowTimer = 0` in `reset()` • `if (this.glowTimer > 0) this.glowTimer--` in `tickTimers()` • `triggerLeafGlow()` method • A new glow branch in `applyHurtBlinkTint()` — hurt blink still takes priority, then the golden pulse, then normal white **`Game.js`** — one new event subscription after the sound block that calls `triggerLeafGlow()` when `leaf:collected` fires The glow is a warm gold (`rgb(255, ~180, 50)`) that pulses using a sine wave and fades out over ~50 frames. You can tune `GLOW_FRAMES` in `PlayerEntity` to make it longer or

	shorter.